DON'T GO BROKE IN THE AGE OF AI

Before the Idea Becomes Reality

[COVER ILLUSTRATION — style-match required]

Where This Booklet Fits: This is booklet 1 of 5 in the Don’t Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

Field Note Opening

Not every idea is ready to cost you money.

You have an idea. You are excited. You want to open an AI builder and start generating screens right now.

Wait.

Before you type a single prompt, read this.

AI has made it incredibly easy to turn vague ideas into visible output. But visible output is not a project. When you let an AI builder start making assumptions about your idea before you have defined it, you are pouring a foundation without knowing what you are building. That is how excitement turns into expensive, unmanageable chaos.

This booklet is a tool to protect your idea from premature execution. It is the first stage in the *Don't Go Broke* series because it focuses on the earliest, most preventable mistake a first-time builder can make: **treating a vague idea as if it is ready to become a real build.**

You do not need a business plan, a pitch deck, or startup jargon. You just need to slow down long enough to answer a few plain questions:

- What is the idea?
- What problem does it solve?
- Who is version one for?
- What counts as version one?
- What is not included yet?
- What would make this project too risky to start right now?

This booklet is not here to kill your idea. It is here to protect it from becoming expensive before it becomes clear.

Read it before you choose a platform. Before the blueprint. Before the build request. Before the idea becomes reality.

Section 1
Introduction: Why This Booklet Exists

A few years ago, an unclear app idea stayed unclear for a while. You might talk about it or sketch it, but there was friction between the idea and the build. Software engineering was expensive, slow, and opaque to non-technical founders. You could not simply wish a database into existence. You had to convince a developer, pay an agency, or spend months learning how to code yourself. That friction was frustrating, but it served a purpose: it forced you to think before you built.

Today, that friction is gone. You can open an AI tool, describe your idea, and watch it generate screens, database assumptions, and features before you have even decided what the project actually is.

That feels amazing. It also makes the idea feel more ready than it really is.

The danger is not that you are excited. You should be excited. The danger is that excitement can turn into a build request before the idea has earned the right to become a project.

Once AI starts building, the project creates gravity. Now there are files. Now there are features to fix and decisions you did not realize you had made. Now stopping feels like waste. That is how a vague idea becomes a real project before you are ready to manage it.

This booklet creates a pause before that happens.

Not a formal planning process. Just enough of a pause to decide what must be true before your idea becomes real.

Before you ask AI to build anything, answer this:

> "What am I about to ask AI to build?"

Because the honest answer may be:

> "I know what I am excited about. But I do not yet know what the project is."

That is not failure. That is the exact moment this booklet is for.

Section 2
The Idea Readiness Audit

Before you can decide if an idea is ready to become a build, you must perform a practical audit of what you actually have. An idea is weightless; a build has gravity. The audit forces you to measure the weight of your idea.

If you skip this audit, you will eventually perform it anyway — but you will do it months from now, when the code is broken, the UI is messy, and you are exhausted trying to figure out why no one wants to use the app.

Do the audit while the idea is still cheap. Ask yourself these baseline questions right now:

1. **What problem does this idea solve?** (If you can only name features, you don't know the problem yet. Features are answers. You need to know the question.)
2. **Who is the idea for?** (If the answer is "everyone," the idea is not ready. Version one needs a specific person.)
3. **What do you expect the app, tool, or system to do?** (What is the core mechanic that makes it valuable? What is the one action the user takes that makes them glad they installed it?)
4. **What result would count as success?** (Is it time saved? Money earned? A frustrating manual task completely automated?)
5. **What could go wrong?** (If you build it perfectly, how could it still fail? What assumptions are you making about user behavior? Will people actually remember to open it?)
6. **What do you not know yet?** (Are you assuming people will pay for this? Are you assuming the necessary data is legally available?)
7. **Does this idea actually need software, or could it be tested manually?** (We will return to this critical question later. It is the most important question in this booklet.)

Write down the answers. If the answers are vague, the idea is vague. That is fine. But it means the idea is not ready to become a project.

Section 3
Why AI Makes Ideas Feel Ready Too Soon

AI can make an idea look real very quickly. You type a description, and a dashboard or a workflow appears. It feels like progress.

But a demo is not proof that the idea deserves to become a real product. A working screen does not prove the problem matters.

AI is excellent at making vague ideas concrete. This helps when you are exploring, but it can trick you into believing the idea is ready just because something visible exists. Once you see it, emotional pressure kicks in. You start reacting to the output: "Add login." "Make it mobile." "Add push notifications."

Now the project is growing. But growing around what?

If the original problem was unclear, every new feature makes the project harder to understand. If version one was never defined, every addition feels reasonable. AI can keep generating, but that does not mean you should keep building.

> **Field Note: The Illusion of Completion**
>
> *An AI can generate a beautiful dashboard with fake charts in ten seconds. It looks done. But fake charts do not require database connections, user authentication, data pipelines, or error handling. The visual surface is 5% of the work. The danger is that looking at the 5% convinces you that you are ready to manage the other 95%.*

A quick demo is a sketch, not a permit to pour the foundation.

Before you trust the excitement of visible progress, ask:

"What would make this idea worth building if AI had not already generated anything?"

This question separates the actual value of your idea from the thrill of seeing something appear. The answer should come from the problem and the user — not from the fact that AI made something quickly.

[ILLUSTRATION — style-match required]

Section 4
The Difference Between an Idea and a Project

An idea says: "*This could be something.*" A project says: "*This is what we are building first.*"

Those are not the same. An idea can be messy, unfinished, and full of possibilities. A project creates obligations. It has scope, files, costs, and things that can break or drift.

Projects are real. Before your idea becomes a project, you need a boundary to protect you from building everything at once.

A project should have answers to four basic questions:

- What problem does this solve?
- Who is version one for?
- What does version one include?
- What is not included yet?

If you cannot answer those questions yet, you have a good idea, but you do not have a project.

This distinction matters because AI does not know the difference. If you give AI a vague idea, it will confidently build a vague project. It will create structure around assumptions you never approved and fill in gaps with patterns that look reasonable but do not match what you actually need.

Once the idea becomes a project, it starts pulling in more decisions.

The question is:

> "Is this still an idea, or have I already started treating it like a project?"

A second question may be even more useful:

> "What have I already committed to before defining the project clearly?"

Maybe you have already committed to a feature, a platform, or a design direction. That is okay. You can still step back. The goal is to notice the difference between imagining and committing, so you can decide whether you are ready to cross that boundary.

Why This Feels Like Overthinking (And Isn't)

The distinction between an idea and a project can feel like an academic exercise when you are excited to start building. It is not. It is the difference between a decision you made on purpose and a decision AI made for you by default.

Here is what that looks like in practice. A founder wants "an app to help freelancers manage their business." That is an idea — broad, exciting, and completely unbuildable as stated. When they open an AI builder and start describing it, the AI has to fill in the gaps immediately: which freelancers, which parts of "managing a business," what the first screen looks like. It will pick something. It always picks something. The question is whether you picked it, or whether the AI picked it for you because you never said otherwise.

A project is what happens when you make those choices on purpose, before the AI has to guess. It does not need to be perfect or permanent. It just needs to exist before the building starts, so that every screen the AI generates is answering a question you actually asked.

Section 5
The Problem Before Product Test

Many app ideas begin as features: "I want an app with profiles, reminders, dashboards, and groups."

That describes what the app contains, but it does not explain what problem it solves. Features are just possible ways to solve a problem. If the problem is unclear, AI can still build those features — leaving you with a lot of visible work wrapped around a vague purpose.

Product-shaped ideas often hide unclear problems. If you start by imagining the product, you assume the problem exists. You must force yourself to describe the problem without naming a product. This is the **Problem Before Product Test**.

Failing the test (Product-shaped thinking):

- "I need an AI meal planning app."
- "I need a dashboard."

Passing the test (Problem-shaped thinking):

- "My family wastes thirty minutes every night deciding what to eat, and we end up ordering expensive takeout."
- "I cannot tell which customers are stuck unless I manually inspect five different spreadsheets."

The Toothbrush Test

A good problem passes the Toothbrush Test: does the user experience this problem every day, or just once a year? If they only experience it once a year, they will not download an app to solve it, no matter how beautiful the UI is. You are building software, which requires habit formation. If the problem is not painful enough or frequent enough to demand a habit, it might not be a problem worth solving with software.

Before your idea becomes a project, write the problem in plain language. Do not try to sound technical or impressive. Try to make it clear.

Use this sentence:

> This idea helps _____ do, avoid, decide, understand, or manage _____ because _____.

Examples:

- This idea helps local bands book gigs because venues take too long to reply to emails.

The sentence does not need to be perfect. It just needs to be clear enough to guide your decisions.

A weaker version sounds like: "*The app has a calendar, an AI assistant, and payment processing.*" That avoids the real question:

> "What problem would still exist if this app did not?"

If you can only describe features, the idea still needs a clearer job.

A useful problem statement gives you a way to judge version one. If the problem is "freelance invoices get lost in email," version one does not need a full business accounting suite. It just needs to help the writer see who hasn't paid yet. The problem protects the project by helping you say yes to what matters and "not yet" to what does not.

Write the simplest version you can:

> "The problem is: _____."

Then ask:

> "Could someone understand this problem without seeing the app?"

Section 6
The Real User Check

Version one is not for everyone.

That can be hard to accept because your idea may eventually help many kinds of people. But "eventually" is not version one. Version one needs a first user.

Not a perfect persona, a market segment, or a startup buzzword. Just a real, specific human being who will interact with what you build.

You need to perform a **Real User Check**. You must identify exactly who is involved in this project, because the "user" is rarely just one person.

Ask yourself:

- **Who will actually use it day-to-day?** (The person clicking the buttons.)
- **Who will approve it?** (If this is for a business, who says "yes, you can use this"?)
- **Who will pay for it?** (The buyer is often not the daily user.)
- **Who will be annoyed by it?** (Whose current workflow are you disrupting?)
- **Who will maintain it?** (When the AI tool breaks, who fixes it? Usually, you.)
- **Who will be blamed if it fails?** (If data is lost, who is responsible?)

If your answer to "Who is this for?" is "everyone," version one has no anchor. Different people need different things, and those different expectations create conflicting screens, settings, and workflows. That is how a small idea becomes bloated before it becomes clear.

A first user protects you from that.

> **Field Note: The Danger of "Me" as the User**
>
> *Many builders build for themselves. This is a great place to start. But even if you are the user, you must separate yourself from the product. You are highly motivated to use the product because you built it. Will the version of you who is tired, busy, and distracted on a Tuesday afternoon actually use it? The "user" is a ruthless editor. Do not let your pride as the builder override the reality of the user.*

Use this sentence:

> The first version is for _____, who needs help with _____.

Examples:

- The first version is for a dog walker who needs help organizing daily client routes.
- The first version is for a beginner gardener who needs help knowing when to water their indoor plants.

These examples do not describe the whole future. They describe the first person the app must help. That first user gives version one a test. If it helps them, version one may be useful. If it does not, more features will not save it.

You may eventually want agencies, commercial farms, or enterprise teams to use the product. But version one only needs to serve one of them. That is a way to keep the first project understandable.

Ask:

> "If this first user is the only person version one helps, what does it need to do well?"

A first version needs someone specific to help. Without that person, the project drifts in every direction at once.

[ILLUSTRATION — style-match required]

Section 7
The Manual Version Test

AI can write code so fast that it is tempting to build software to solve every problem. But software is expensive to maintain, hard to change, and prone to breaking.

Before you build software, you must run the **Manual Version Test**.

Can you solve this problem manually right now? If you cannot create value manually, software will probably not fix the idea. Software only scales existing value; it does not invent it.

Examples of Manual Versions:

- **Spreadsheet:** Instead of building a complex database app, can you track the data in Google Sheets first?
- **Checklist:** Instead of building an automated workflow engine, can you give the user a simple PDF checklist?
- **Text message workflow:** Instead of building a chat interface, can you just text your first three users manually?
- **Email process:** Instead of building a notification system, can you send the emails yourself?
- **Paper form:** Instead of building a web form, can you ask the questions on a clipboard?
- **Airtable or Notion mock process:** Can you use an existing no-code tool to string the data together before writing custom code?

The Concierge Test

This is the most powerful manual test. You pretend to be the software.

If your app is supposed to curate weekly meal plans based on dietary restrictions, do not write code. Instead, text a friend on Sunday and ask what they like to eat. Then, spend an hour manually researching recipes. Email them a meal plan PDF on Monday morning.

If they don't open the email, or if they don't cook the meals, you have just learned that the app would have failed. You learned this without spending a single dollar on

hosting, without debugging an AI agent, and without building a complicated web interface. You learned it for the cost of an hour of your time.

If a user will not pay you or thank you when you do the work manually, they will not care when an AI does it automatically.

Test the idea manually first. If it works, you have proven the problem exists and the solution is valuable. *Then* you are ready to build software.

When the Manual Version Fails, That's Useful Too

Do not treat a failed manual test as a wasted afternoon. It is the cheapest version of failure you will ever get.

If you text three people your "concierge" meal plan and none of them cook it, you just learned that the value you assumed was there is not — before spending a single dollar on AI credits, hosting, or a database schema. Compare that to launching a real app, waiting weeks for signups, and discovering the same thing after you have already built authentication, a payments integration, and a dashboard nobody opens.

The manual version test is not a hurdle standing between you and building software. It is the fastest, cheapest way to find out whether the software is worth building at all. Founders who skip it are not saving time — they are postponing the same discovery until it costs ten times as much to make.

Section 8
What Counts as Version One?

Version one is not the whole dream. It is the smallest useful version of the project.

That word matters: **useful.** Not impressive. Not complete. Not future-proof. Useful.

Version one should solve one clear problem for one clear first user. It should be small enough that you can understand it, review it, and control it.

If version one is too large, you may not know whether AI built the right thing. You may not understand which feature caused which bug, or whether the project is drifting away from the original idea. A smaller version one gives you a better chance to stay in control.

Use this sentence:

> The first useful version only needs to let the user _____.

Examples:

- The first useful version only needs to let the dog walker see today's route on a map.
- The first useful version only needs to let the freelance writer log a sent invoice and click a button when it is paid.

These are not complete products, and that is the point. A first room is not the whole building, but it can prove that the structure stands.

> "If this one thing worked, would version one be useful enough to learn from?"

If the answer requires ten more features, version one may be too large. Every added feature increases the load. More features mean more opportunities for AI to guess, which means more places for cost and confusion to enter.

Every feature should earn its place. Before the build starts, version one should be something you can explain in one or two sentences.

Section 9
What Is Explicitly Not Included Yet?

A good version one needs a "not yet" list.

When you are excited, every future feature feels important: accounts, social sharing, payments, mobile push notifications, AI recommendations, analytics. Maybe you will need those. But not all at once, and maybe not in version one.

"Not yet" is not rejection. It is protection. It lets you keep an idea without letting it invade the first build.

Use this sentence:

> For version one, we are not building _____, _____, or _____ yet.

Examples:

- For version one, we are not building customer portals, automatic route optimization, or billing yet.
- For version one, we are not building multi-user teams, Stripe integration, or custom email templates yet.

The "not yet" list is a parking lot, not a trash can. You are parking ideas where they cannot drive the first version off the road.

This list also protects you when you work with AI. If you do not tell AI what is excluded, it may assume accounts, settings pages, and admin dashboards are needed just because they are normal in similar apps. A "not yet" list keeps those assumptions out of your foundation.

Ask:

> "Which future feature is most likely to sneak back into version one?"

Write it down. That is the feature to watch. If everything feels too important to exclude, the project boundary is not strong enough yet.

[ILLUSTRATION — style-match required]

Section 10
The Cost of Making It Real

Before an idea becomes a project, it exists only in your mind. It is free. It has no bugs, no hosting costs, and no angry users.

The moment you start building, you cross a threshold. Making an idea real creates obligations. AI makes the building part cheap, but it does not make the owning part cheap. You need to understand the **Cost of Making It Real**.

Once your app exists, you must deal with:

- **Accounts:** Users will forget their passwords. Who resets them? Do you have an automated flow, or are they emailing you?
- **Data:** Where is the data stored? Is it safe? What happens if the database is accidentally deleted?
- **Permissions:** Who is allowed to see what? If user A can see user B's data, you have a major security problem.
- **Support:** When the app breaks on a Friday night, who gets the email? Are you prepared to fix a bug while on vacation?
- **Maintenance Debt:** AI tools update. APIs change. Libraries deprecate. Software rots if left alone. A project you build today will require maintenance next year just to keep running.
- **Security Obligations:** If you collect user data, you are legally and ethically responsible for it.
- **User Confusion:** People will use your app in ways you never intended. They will upload huge files, click buttons twice, and ignore the instructions.
- **Platform Costs:** Hosting, database reads, API calls, domain names. These start small but scale quickly.
- **The Cost of Attention:** Every minute you spend fixing a bug is a minute you are not spending living your life, doing your actual job, or building the next feature.
- **The Cost of Abandonment:** What happens to the users if you get bored and shut the app down? Do they lose their data? Do you have an export feature?

A Founder Who Skipped This List

A builder launched a scheduling app for local tutors in a single weekend, thrilled at how fast AI had made it real. Three months later, a parent's account was hacked because there was no password-reset flow — just a support email address nobody was checking. A tutor uploaded a twelve-hour audio file that silently ate through the month's storage budget. And when the founder finally got busy with a new job and stopped logging in, the app kept running, kept billing, and kept holding two hundred families' data with nobody responsible for any of it.

None of this happened because the AI wrote bad code. It happened because nobody had asked, before building, who resets a password, who owns the data, or what happens if the founder walks away. The build was fast. The obligations were still there, waiting, whether or not anyone had planned for them.

This is not a reason to avoid building. It is a reason to look at this list honestly, before the app exists, while every answer is still just a decision instead of an emergency.

Do not let this list scare you. Let it clarify your intent.

If you understand these obligations and are willing to take them on, you are ready to build. If looking at this list makes you exhausted, your idea might be better suited as a manual test, a spreadsheet, or simply an idea for later.

Section 11
What Would Make This Project Too Risky?

Some ideas should not become projects yet. That does not mean they should die; it means they need more clarity before they become real.

A stop sign tells you where the risk is before AI starts generating files and features around it. The goal is not to scare you, but to help you build with your eyes open.

Use this sentence:

> This project is not ready if _____.

Instead of a long checklist, look for these practical warning lights. A warning light does not mean the car is worthless, but it does mean you should not keep driving as if nothing is wrong.

Stop Sign 1: You cannot explain the problem simply If you need a long explanation before someone understands the problem, the project is too vague. If you cannot fill in "The problem is: _____" cleanly, pause.

Stop Sign 2: You cannot name the first user If version one is for "everyone," it will probably become too large. If you cannot name the first person who needs this, pause.

Stop Sign 3: Version one has too many jobs A first version should not solve five problems at once. If it needs to manage, recommend, schedule, analyze, and sell, it is too large.

Stop Sign 4: You cannot say what is not included yet If nothing can be excluded, the project has no boundary.

Stop Sign 5: The project depends on features you do not understand You do not need to be a professional developer. But if version one depends on things you cannot explain, review, or recognize when broken, the project may be too risky. Version one may need to be smaller.

Stop Sign 6: You cannot review whether AI built the right thing If AI gives you output and you would not know whether it matches the intended project, pause. If the project is too vague to review, it is too vague to build.

Stop Sign 7: You cannot explain what "done" means A project needs an end point for version one. If you cannot say what counts as a usable first version, AI may keep building past the point where you understand the project.

A stop sign means the idea needs more clarity before it becomes real.

Using the Stop Signs Without Freezing

Seven stop signs can feel discouraging if you read them looking for reasons to quit. That is not what they are for.

A stop sign is not a verdict. It is a warning light on a dashboard — it tells you where to look, not that the car is broken beyond repair. Most ideas will trigger at least one or two of these signs on the first pass, and that is normal, not a failure.

The difference between a stop sign and a dead end is what you do next. If Stop Sign 2 lights up because you cannot name the first user, the fix is not to abandon the idea — it is to go find that user before you build anything. If Stop Sign 7 lights up because you cannot explain what "done" means, the fix is to write that definition down before you ask AI for a single screen.

Treat each lit warning light as a task, not a diagnosis. Resolve it, then check again. An idea that clears all seven signs after some real work is in a much stronger position than one that looked ready on the first pass but was never actually tested against them.

Section 12
The "Not Yet" Decision

In a world where you can build anything instantly, the most powerful decision you can make is "Not Yet."

"Not Yet" is a valid success. It is the sound of a builder protecting their time, their budget, and their sanity.

If your idea hits a stop sign, or if the cost of making it real outweighs the value of the manual version, you have several excellent options that do not involve asking an AI to write code:

- **Park the idea:** Write it down in a notebook and walk away. Let it simmer. Return to it when you have more clarity.
- **Narrow the idea:** Cut the scope in half. Then cut it in half again. Find the absolute smallest sliver of value.
- **Test manually:** Run the Concierge test. Do it yourself with a spreadsheet. Serve one person perfectly.
- **Research first:** Go talk to three real people who actually have the problem. Don't mention your app idea. Just ask them how they solve the problem today.
- **Define the user first:** Stop guessing who will use it and go find them.
- **Write a better problem statement:** Strip away the product features until the raw problem remains.
- **Wait until a real use case appears:** Sometimes an idea is great, but the timing is wrong.

Building software is not the only way to make progress. Sometimes, the best progress is deciding not to build software today.

[ILLUSTRATION — style-match required]

Section 13
Ready, Pause, or Safe First Commitment

You do not need certainty. You just need enough clarity to move forward with your eyes open.

At this stage, there are three useful decisions: Ready, Pause, or Safe First Commitment. All three can be good decisions.

Ready

The idea may be ready to become a project if you can clearly answer the core questions: the problem, the first user, the version-one boundary, the "not yet" list, and the stop signs.

Ready does not mean the project will succeed, nor does it mean you have solved every future problem. Ready simply means the idea has enough shape that you can protect it. It is clear enough to become a controlled project.

Pause

Pause may be the right decision if the idea is promising but one or more basic decisions are still unclear. Maybe the problem is almost clear, or you know the user but not version one. Maybe the "not yet" list is missing.

Pause is not failure. Pause is how you avoid building confusion into the foundation.

Safe First Commitment

If the idea is ready, your next move is not to build the entire app. Your next move is to choose the smallest **Safe First Commitment**.

A Safe First Commitment is an action that moves the project forward without trapping you in an expensive build cycle.

Examples of a Safe First Commitment:

- **Write a one-page concept summary:** Distill the problem, the user, and the version one boundary into a single page of plain text.
- **Talk to one likely user:** Show them a sketch or ask them about how they solve the problem today.

- **Map out one single workflow on paper:** Draw boxes and arrows. Do not open software. Just map the logic.
- **Manually run the process once:** Do the concierge test for a single person.
- **Collect five examples:** Find five similar tools and note exactly what you dislike about them.
- **Create the formal Build Brief:** Formalize your answers into a document you can hand to an AI.

Before you move forward, make the call:

> My decision is: Ready / Pause / Safe First Commitment
>
> Because: _____.

If the answer is Ready, there is one more important thing to know. The next step is still not "ask AI to build it."

Section 14
Bridge to the Next Booklet: Before the Build Starts

If your idea is ready to become a project, you have done important work. You have defined the idea, the problem, the first user, version one, what is not included yet, and the stop signs.

That crosses the first gate — but it is not yet enough to hand the project to an AI agent and let it start generating code.

This booklet helped you decide whether the idea deserves to become a real project. You have defined the problem, located the user, and established the boundaries of version one.

The next job is to design the project before AI begins building. That is where the next booklet, *Before the Build Starts*, comes in.

Before the Build Starts deals with the blueprint stage. It will help you define what screens are needed, what user flows must be mapped, what data is involved, what constraints matter, and how to create a handoff that protects the project from AI hallucinations.

Those are not the questions for this booklet. It ends at the moment the idea earns project status. If it has, carry these answers forward:

- This is the problem.
- This is who version one is for.
- This is what version one includes.
- This is what is not included yet.
- This is what would make the project unsafe to start.
- This is the safest next commitment.
- This is why the idea is ready to become a project.

Then design before you build.

You are not ready to build just because you are excited.

Your job is to make the project clear enough to protect. You are ready to build when the project is clear enough to protect.

[ILLUSTRATION — style-match required]

Field Note Closing

The build will feel urgent. It always does.

Before you opened this booklet, the excitement was the only thing in the room. Now something else is in the room with it: a clear problem, a first user, a version-one boundary, and a list of what is not ready yet.

That is not just planning. That is protection. An idea with those four things is harder to lose and harder to mishandle. You can explain it. You can review it. You can hand it to an AI and recognize when it drifts.

Most builders never get this pause. They open the AI tool, describe the excitement, and watch the excitement turn into files before they have decided anything on purpose. Then they spend the next six months discovering, one broken assumption at a time, everything they should have decided in the first afternoon.

You did it differently. You made yourself answer plain questions before asking anything of the AI: what the problem actually is, who actually needs it solved, what the smallest useful version looks like, and what you are deliberately choosing not to build yet. That is not caution for its own sake. It is the difference between directing a fast tool and being dragged along by one.

None of this guarantees the project will succeed. Nothing does. What it guarantees is that if something goes wrong, you will be able to say exactly what you meant to build, and compare that against what actually got built. That comparison is the entire game. Most expensive AI-assisted failures are not failures of the tool. They are failures of nobody having written down, in advance, what "right" was supposed to look like.

The next booklet will ask you to design the project before you let AI start building it. Bring what you found here. It is enough to cross the first gate.

The urgency will return. Let it. You are better prepared for it now.

Back Matter

Idea Readiness Audit Worksheet

Use this worksheet before treating an idea like a project.

1. The Idea

The idea is:

2. The Problem Before Product

This idea helps _____ do, avoid, decide, understand, or manage _____ because _____.

Or:

The raw problem is:

3. The Real User Check

The first version is for _____, who needs help with _____.

Who will actually use it day-to-day?

Who will approve it?

Who will maintain it?

4. The Manual Version Test

Can this be solved manually first? (Spreadsheet, email, concierge)

If yes, how will we test it?

If no, why is software absolutely required today?

5. Version One Boundary

The first useful version only needs to let the user _____.

6. Not Yet List

For version one, we are not building:

1.
2.
3.

7. Stop Signs & Costs

This project is not ready if:

- The problem is still vague.
- The first user is unclear.
- Version one has too many jobs.
- I cannot say what is not included yet.
- The project depends on features I do not understand.
- I cannot review whether AI built the right thing.
- I cannot explain what would make version one done.

8. Safe First Commitment

My next safe step is:

9. Decision

My decision is: Ready / Pause / Safe First Commitment

Because:

One-Page "Not Yet" List

A "not yet" feature is not rejected forever — it is excluded from version one so the first project stays clear and controllable.

Not included yet:

Why it is not included in version one:

When it might be reconsidered:

Not included yet:

Why it is not included in version one:

When it might be reconsidered:

Not included yet:

Why it is not included in version one:

When it might be reconsidered:

Next Step

If your decision is Ready, and you have completed the Safe First Commitment, continue to the next booklet:

Before the Build Starts

Bring your Idea Readiness Audit with you. The next step is not to let AI build freely. The next step is to draw the blueprint before the build starts.

[NEXT BOOKLET COVER — insert when available]